

Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
In-Class Exercise (11/05)

- Consider a MIPS system with 32-bit virtual addresses, 24-bit physical addresses and a page size of 4KB. The processor has a 4-entry fully-associative **TLB** and a 2-way set-associative **data cache** with 64B capacity and 16B block size, the contents of which are shown below. The cache is **physically-indexed physically-tagged (PIPT)**.

TLB

Tag	Page Table Entry (PPN)
0x20C80	0x711
0x10800	0x20F
0x04122	0x030
0x50840	0x144

Data Cache

Set	Tag	Data Words in Block			
		Offset 0	Offset 4	Offset 8	Offset 12
0	0x2F720	0x77777777	0xBBBBBBBB	0xFFFFFFFF	0x33333333
	0x03044	0xAAAAAAAA	0xEEEEEEEE	0x22222222	0x66666666
1	0x10790	0xDDDDDDDD	0x11111111	0x55555555	0x99999999
	0x08220	0x00000000	0x44444444	0x88888888	0xCCCCCCCC

What are the **contents of register \$s0** after running the following instructions on this processor? Provide your answer in hexadecimal.

```
lui $s1, 0x1080
ori $s1, $s1, 0x0218
lw  $s0, 0($s1)           # [$s1] = 0x10800218
```

Answer:

TLB hit at set 0 with tag 0x10800. Translation: VA 0x10800218 → PA 0x20F218.

Data cache hit at set 1 with tag 0x10790. Return data word at offset 8.

Thus [\$s0] = 0x55555555.

2. Consider a processor with a **virtually-indexed physically-tagged (VIPT) cache**. The cache capacity is 64KB with a block size of 128B. The page size in the system is 4KB. You use the standard set indexing scheme on the virtual address, but the tag comes from the physical address as shown:

Virtual Address:

	set index	block offset
--	-----------	--------------

Physical Address:

tag	block offset
-----	--------------

VIPT caches may be prone to the **synonym** problem. Recall that synonyms are virtual addresses that map to the same physical address. If unhandled, a physical block may wrongfully exist in the cache at **more than one location** at any time, which leads to incorrect behavior (i.e., one instance of the block becomes stale if the other instance is modified).

- a) Assume the VIPT cache is **8-way** set-associative. You run two different programs on this processor (Program A and Program B) that want to map the **same** memory block at **physical address** 0x0100C800 into each of their virtual address spaces. Give an example **pair of virtual addresses** (one for Program A and one for Program B) that, when mapped to physical block 0x0100C800, can wrongfully place this block into two unique locations in the VIPT cache. Answer in hexadecimal and explain your answer.

Answer:

Since the set index bits (bits 12 to 7) are not completely within the page offset bits (bits 11 to 0), then any pair of virtual addresses that differ at bit 12 would have the synonym problem.

Example: virtual addresses 0x00000800 and 0x00001800.

- b) Without changing the VIPT cache capacity (64KB), block size (128B) and page size (4KB), what is the **minimum associativity** needed such that synonyms would **not** be a problem?

Answer:

Maximum number of sets = 4KB pages / 128B blocks = 32

Thus minimum associativity = (64KB cache / 128B blocks) / 32 = 16 ways

3. You plan to support **single error correction** for all 4-bit chunks of data in memory. You are considering three possible encoding schemes, shown below. Each scheme converts 4-bit data ($b_4b_3b_2b_1$) to a **codeword** ($\dots W_3W_2W_1$) that is stored in physical memory. Note the 1-based indexing (i.e., least-significant bit is position 1), as per the video lectures.

Scheme A: 8-bit Codeword

W_8	W_7	W_6	W_5	W_4	W_3	W_2	W_1
b_4	b_4	b_3	b_3	b_2	b_2	b_1	b_1

Scheme B: 12-bit Codeword

W_{12}	W_{11}	W_{10}	W_9	W_8	W_7	W_6	W_5	W_4	W_3	W_2	W_1
b_4	b_4	b_4	b_3	b_3	b_3	b_2	b_2	b_2	b_1	b_1	b_1

Scheme C: 7-bit Codeword

W_7	W_6	W_5	W_4	W_3	W_2	W_1
			b_2		b_1	b_1
			$\oplus$		$\oplus$	$\oplus$
b_4	b_3	b_2	b_3	b_1	b_3	b_2
			$\oplus$		$\oplus$	$\oplus$
			b_4		b_4	b_4

- a) You implement **Scheme A** and run your processor. After some time, you observe memory and find a codeword with value 11001000 ($W_8W_7W_6W_5W_4W_3W_2W_1$). Assuming at most **a single bit has flipped** in this codeword, what are all the possible values for the **original 4-bit data** ($b_4b_3b_2b_1$) encoded by this codeword?

Answer:

$$W_4 \neq W_3$$

Thus 1010 or 1000; single error correction is not possible

- b) You implement **Scheme B** instead and find a codeword with value 000101111000 ($W_{12}W_{11}W_{10}\dots W_3W_2W_1$). Assuming at most **a single bit has flipped**, what are all the possible values for the **original 4-bit data** ($b_4b_3b_2b_1$) encoded by this codeword?

Answer:

$$W_8 \neq (W_9 = W_7 = 1)$$

Thus 0110

- c) You implement **Scheme C** instead and find a codeword with value 1101100 ($W_7W_6W_5W_4W_3W_2W_1$). Assuming at most **a single bit has flipped**, what are all the possible values for the **original 4-bit data** ($b_4b_3b_2b_1$) encoded by this codeword?

Answer:

$$W_7 \oplus W_5 \oplus W_3 \oplus W_1 = 0 \text{ and } W_7 \oplus W_6 \oplus W_3 \oplus W_2 = 1 \text{ and } W_7 \oplus W_6 \oplus W_5 \oplus W_4 = 1$$

Thus 1001